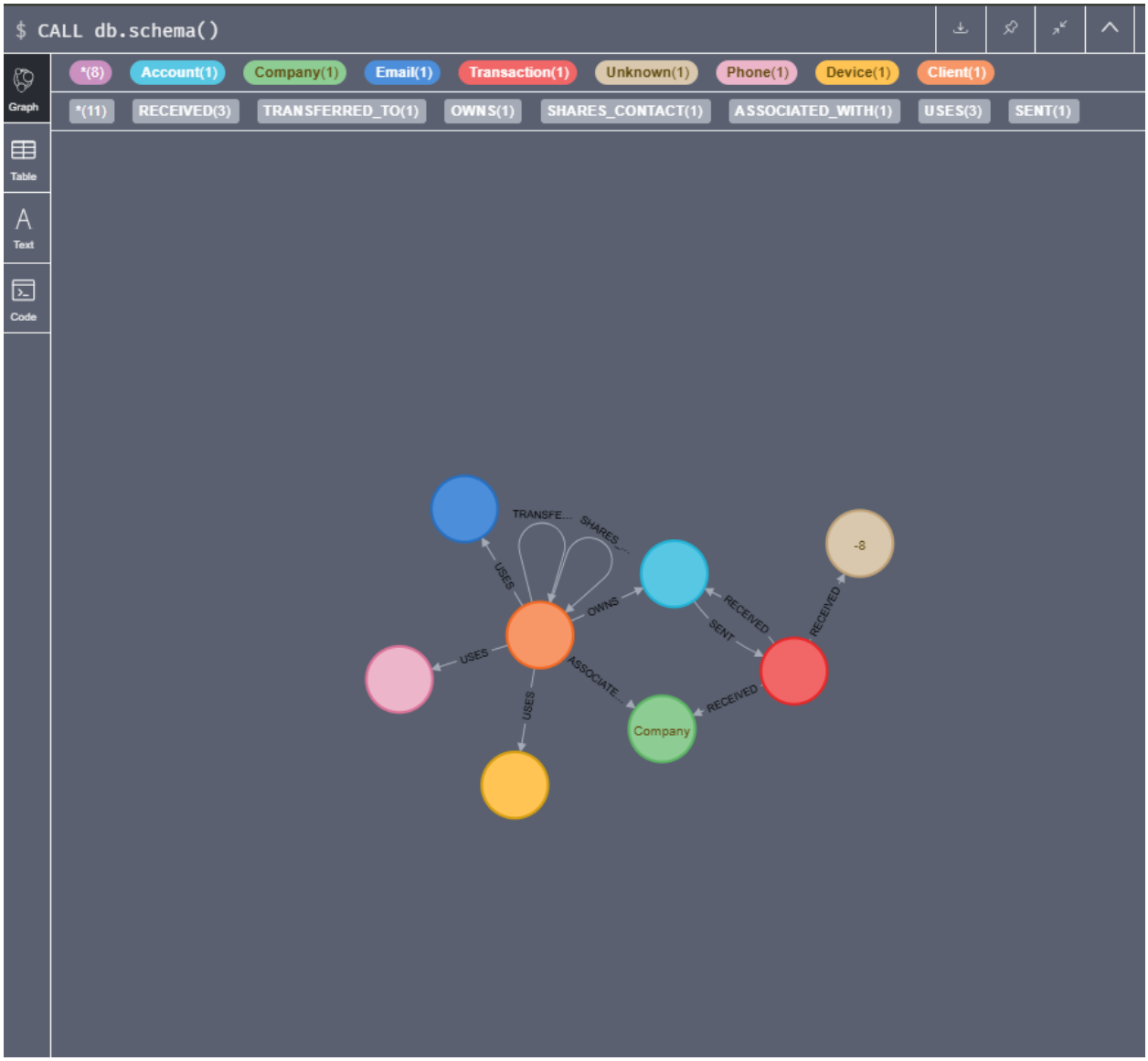
Neo4j Запросы

1) Проверка узлов и связей (тк данные уже загружены, то не пусто)

CALL db.schema();

Или

CALL db.schema.visualization();



2) Вызов – проверка количества узлов каждого типа

MATCH (c:Client) RETURN 'Clients' as Type, count(*) as Count

UNION ALL

MATCH (a:Account) RETURN 'Accounts' as Type, count(*) as Count

UNION ALL

```
MATCH (comp:Company) RETURN 'Companies' as Type, count(*) as Count
UNION ALL
MATCH (t:Transaction) RETURN 'Transactions' as Type, count(*) as Count
UNION ALL
MATCH (p:Phone) RETURN 'Phones' as Type, count(*) as Count
UNION ALL
MATCH (e:Email) RETURN 'Emails' as Type, count(*) as Count
UNION ALL
MATCH (d:Device) RETURN 'Devices' as Type, count(*) as Count
ORDER BY Count DESC;
```

$ MATCH (c:Client) RETURN 'Clients' as Type, count(*) as Count UNION ALL MATCH …

Table | Text | Code

Type	Count
"Clients"	35000
"Accounts"	102679
"Companies"	15000
"Transactions"	600000
"Phones"	68105
"Emails"	51439
"Devices"	55770

3) Загрузка основных данных из внутренней папки import с разбиением больших данных на более маленькие подгрузки

```
//Загрузка клиентов
:auto USING PERIODIC COMMIT 5000
LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row
CREATE (c:Client {
    client_id: toInteger(row.client_id),
    fio: row.fio,
    birth_year: toInteger(row.birth_year),
    registration_region: row.registration_region,
    client_tier: row.client_tier,
    risk_score: toFloat(row.risk_score),
    accounts_count: toInteger(row.accounts_count)
```

```
});

//Загрузка счетов
:auto USING PERIODIC COMMIT 5000
LOAD CSV WITH HEADERS FROM 'file:///accounts.csv' AS row
CREATE (a:Account {
    account_id: row.account_id,
    client_id: toInteger(row.client_id),
    account_number: row.account_number,
    account_type: row.account_type,
    open_date: date(row.open_date),
    balance: toFloat(row.balance),
    currency: row.currency,
    status: row.status,
    risk_score: toFloat(row.risk_score)
});

//Загрузка компаний
:auto USING PERIODIC COMMIT 5000
LOAD CSV WITH HEADERS FROM 'file:///companies.csv' AS row
CREATE (comp:Company {
    company_id: row.company_id,
    name: row.name,
    activity_code: row.activity_code,
    activity_name: row.activity_name,
    inn: row.inn,
    ogrn: row.ogrn,
    owner_id: toInteger(row.owner_id),
    risk_score: toFloat(row.risk_score),
    registration_date: date(row.registration_date)
});

//Загрузка транзакций
:auto USING PERIODIC COMMIT 5000
```

```
LOAD CSV WITH HEADERS FROM 'file:///transactions.csv' AS row
CREATE (t:Transaction {
    transaction_id: row.transaction_id,
    from_client_id: toInteger(row.from_client_id),
    from_account_id: row.from_account_id,
    to_id: row.to_id,
    to_type: row.to_type,
    transaction_type: row.transaction_type,
    amount: toFloat(row.amount),
    tx_datetime: datetime(replace(row.tx_datetime, ' ', 'T')),
    tx_hour: toInteger(row.tx_hour),
    tx_day_of_week: toInteger(row.tx_day_of_week),
    frequency_24h: toInteger(row.frequency_24h),
    is_fraud: toInteger(row.is_fraud),
    fraud_score: toFloat(row.fraud_score),
    status: row.status,
    sender_phone: row.sender_phone,
    sender_email: row.sender_email,
    sender_device: row.sender_device
});
```

4) Создание ограничений и индексов для дальнейшего ускорения загрузки и создания связей

```
// Уникальные ограничения
CREATE CONSTRAINT ON (c:Client) ASSERT c.client_id IS UNIQUE;
CREATE CONSTRAINT ON (a:Account) ASSERT a.account_id IS UNIQUE;
CREATE CONSTRAINT ON (comp:Company) ASSERT comp.company_id IS UNIQUE;
CREATE CONSTRAINT ON (t:Transaction) ASSERT t.transaction_id IS UNIQUE;
CREATE CONSTRAINT ON (p:Phone) ASSERT p.number IS UNIQUE;
CREATE CONSTRAINT ON (e:Email) ASSERT e.address IS UNIQUE;
CREATE CONSTRAINT ON (d:Device) ASSERT d.device_id IS UNIQUE;

// Индексы для часто используемых полей
CREATE INDEX ON :Client(risk_score);
CREATE INDEX ON :Account(risk_score);
```

```
CREATE INDEX ON :Account(client_id);
CREATE INDEX ON :Company(risk_score);
CREATE INDEX ON :Transaction(is_fraud);
CREATE INDEX ON :Transaction(tx_datetime);
```

5) Создание узлов для email, телефонов и устройств (обработка построчно)

```
// Узлы для телефонов
:auto USING PERIODIC COMMIT 500
LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row
WITH row WHERE row.phones IS NOT NULL AND row.phones <> '[]' AND row.phones <> ''
WITH row, split(substring(row.phones, 1, size(row.phones)-2), '}, {') as phone_strings
UNWIND phone_strings as phone_str
WITH row, phone_str,
    case when phone_str contains 'number' then true else false end as has_number
WHERE has_number
WITH row,
    replace(replace(replace(phone_str, '{', ''), '}', ''), '"', '') as clean_str
WITH row, split(clean_str, ', ') as parts
WITH row,
    [p in parts where p contains 'number'] as number_part,
    [p in parts where p contains 'type'] as type_part,
    [p in parts where p contains 'is_shared'] as shared_part
WITH row,
    case when size(number_part) > 0 then substring(number_part[0], 9, size(number_part[0])-9) else null end as number,
    case when size(type_part) > 0 then substring(type_part[0], 7, size(type_part[0])-7) else null end as type,
    case when size(shared_part) > 0 then substring(shared_part[0], 12, size(shared_part[0])-12) else 'false' end as is_shared
WHERE number IS NOT NULL
MERGE (p:Phone {number: number})
ON CREATE SET
   p.type = type,
```

```
    p.is_shared = is_shared;


// Узлы для email
:auto USING PERIODIC COMMIT 500
LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row
WITH row WHERE row.emails IS NOT NULL AND row.emails <> '[]' AND row.emails <> ''
WITH row, split(substring(row.emails, 1, size(row.emails)-2), '}, {') as email_strings
UNWIND email_strings as email_str
WITH row, email_str,
    case when email_str contains 'address' then true else false end as has_address
WHERE has_address
WITH row,
    replace(replace(replace(email_str, '{', ''), '}', ''), '"', '') as clean_str
WITH row, split(clean_str, ', ') as parts
WITH row,
    [p in parts where p contains 'address'] as address_part,
    [p in parts where p contains 'domain'] as domain_part,
    [p in parts where p contains 'is_shared'] as shared_part
WITH row,
    case when size(address_part) > 0 then substring(address_part[0], 10, size(address_part[0])-10) else
null end as address,
    case when size(domain_part) > 0 then substring(domain_part[0], 9, size(domain_part[0])-9) else null
end as domain,
    case when size(shared_part) > 0 then substring(shared_part[0], 12, size(shared_part[0])-12) else 'false'
end as is_shared
WHERE address IS NOT NULL
MERGE (e:Email {address: address})
ON CREATE SET
    e.domain = domain,
    e.is_shared = is_shared;


// Узлы для устройств
:auto USING PERIODIC COMMIT 500
LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row
WITH row WHERE row.devices IS NOT NULL AND row.devices <> '[]' AND row.devices <> ''
```

```
WITH row, split(substring(row.devices, 1, size(row.devices)-2), '}, {') as device_strings
UNWIND device_strings as device_str
WITH row, device_str,
    case when device_str contains 'device_id' then true else false end as has_id
WHERE has_id
WITH row,
    replace(replace(replace(device_str, '{', ''), '}', ''), '"', '') as clean_str
WITH row, split(clean_str, ', ') as parts
WITH row,
    [p in parts where p contains 'device_id'] as id_part,
    [p in parts where p contains 'device_type'] as type_part,
    [p in parts where p contains 'os'] as os_part,
    [p in parts where p contains 'is_shared'] as shared_part
WITH row,
    case when size(id_part) > 0 then substring(id_part[0], 11, size(id_part[0])-11) else null end as device_id,
    case when size(type_part) > 0 then substring(type_part[0], 14, size(type_part[0])-14) else null end as device_type,
    case when size(os_part) > 0 then substring(os_part[0], 5, size(os_part[0])-5) else null end as os,
    case when size(shared_part) > 0 then substring(shared_part[0], 12, size(shared_part[0])-12) else 'false' end as is_shared
WHERE device_id IS NOT NULL
MERGE (d:Device {device_id: device_id})
ON CREATE SET
   d.device_type = device_type,
   d.os = os,
   d.is_shared = is_shared;
```

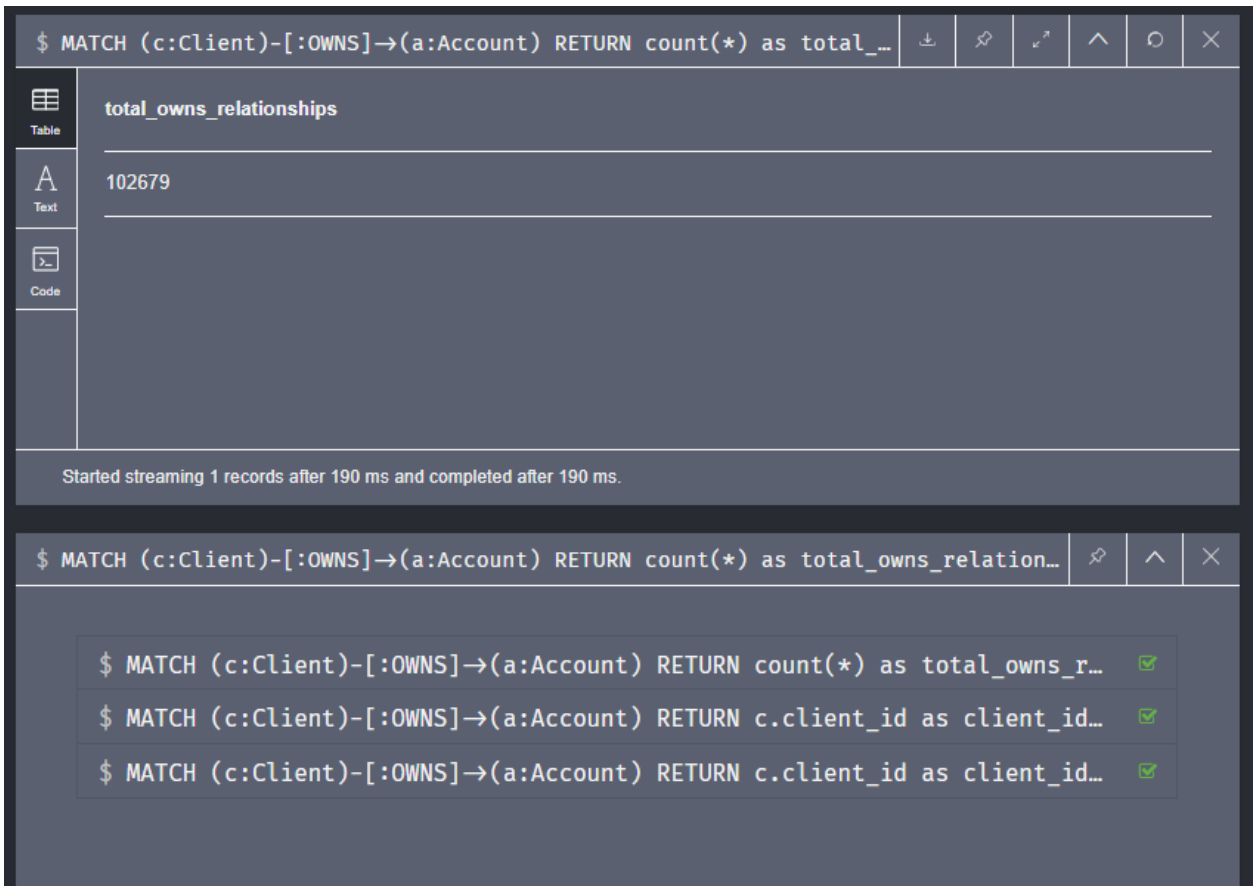
6) Связи принадлежности

```
// Client -> owns -> Account
CREATE INDEX ON :Account(client_id);

:auto USING PERIODIC COMMIT 1000
MATCH (c:Client)
```

```
MATCH (a:Account {client_id: c.client_id})
MERGE (c)-[:OWNS]->(a)
RETURN count(*) as relationships_created;

// Client -> associated_with -> Company (через owner_id)
:auto USING PERIODIC COMMIT 1000
LOAD CSV WITH HEADERS FROM 'file:///companies.csv' AS row
WITH row WHERE row.owner_id IS NOT NULL AND row.owner_id <> ''
MATCH (c:Client {client_id: toInteger(row.owner_id)})
MATCH (comp:Company {company_id: row.company_id})
MERGE (c)-[:ASSOCIATED_WITH {role: 'owner'}]->(comp)
RETURN count(*) as relationships_created;
```

Проверка созданных связей

```
// общее количество связей OWNS
MATCH (c:Client)-[:OWNS]->(a:Account)
RETURN count(*) as total_owns_relationships;

// сколько счетов у каждого
MATCH (c:Client)-[:OWNS]->(a:Account)
RETURN c.client_id as client_id, c.fio as fio, count(a) as accounts_count
ORDER BY accounts_count DESC
LIMIT 20;

// примеры связей
MATCH (c:Client)-[:OWNS]->(a:Account)
RETURN c.client_id as client_id, a.account_id as account_id, a.account_type as account_type
LIMIT 10;
```

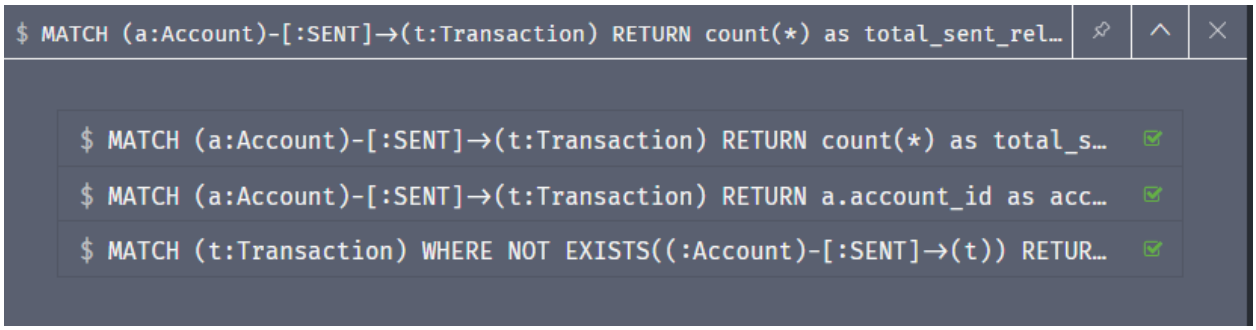


7) Создание транзакционных связей

```
// Account -> sent -> Transaction
CREATE INDEX ON :Transaction(from_account_id);

:auto USING PERIODIC COMMIT 1000
MATCH (t:Transaction)
MATCH (a:Account {account_id: t.from_account_id})
MERGE (a)-[:SENT]->(t)
RETURN count(*) as relationships_created;

// Transaction -> received -> Account (для client)
:auto USING PERIODIC COMMIT 1000
MATCH (t:Transaction)
WHERE t.to_type = 'client'
MATCH (a:Account {account_id: t.to_id})
MERGE (t)-[:RECEIVED]->(a)
RETURN count(*) as client_received;
```

```
// Transaction -> received -> Company
:auto USING PERIODIC COMMIT 1000
MATCH (t:Transaction)
WHERE t.to_type = 'company'
MATCH (comp:Company {company_id: t.to_id})
MERGE (t)-[:RECEIVED]->(comp)
RETURN count(*) as company_received;

// Узлы Unknown для неизвестных получателей
:auto USING PERIODIC COMMIT 1000
MATCH (t:Transaction)
WHERE t.to_type = 'unknown'
MERGE (u:Unknown {id: t.to_id})
MERGE (t)-[:RECEIVED]->(u)
RETURN count(*) as unknown_received;
```

Проверка после создания



8) Создание контекстных связей (общие телефоны, email, устройства)

```
// Client -> uses -> Phone
:auto USING PERIODIC COMMIT 500
LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row
WITH row WHERE row.phones IS NOT NULL AND row.phones <> '[]' AND row.phones <> ''
WITH row, split(substring(row.phones, 1, size(row.phones)-2), '}, {') as phone_strings
UNWIND phone_strings as phone_str
WITH row, phone_str,
```

```
    case when phone_str contains 'number' then true else false end as has_number
WHERE has_number
WITH row,
    replace(replace(replace(phone_str, '{', ''), '}', ''), '"', '') as clean_str
WITH row, split(clean_str, ', ') as parts
WITH row,
    [p in parts where p contains 'number'] as number_part
WITH row,
    case when size(number_part) > 0 then substring(number_part[0], 9, size(number_part[0])-9) else null
end as number
WHERE number IS NOT NULL
MATCH (c:Client {client_id: toInteger(row.client_id)})
MATCH (p:Phone {number: number})
MERGE (c)-[:USES]->(p)
RETURN count(*) as phone_relationships;


// Client -> uses -> Email
:auto USING PERIODIC COMMIT 500
LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row
WITH row WHERE row.emails IS NOT NULL AND row.emails <> '[]' AND row.emails <> ''
WITH row, split(substring(row.emails, 1, size(row.emails)-2), '}, {') as email_strings
UNWIND email_strings as email_str
WITH row, email_str,
    case when email_str contains 'address' then true else false end as has_address
WHERE has_address
WITH row,
    replace(replace(replace(email_str, '{', ''), '}', ''), '"', '') as clean_str
WITH row, split(clean_str, ', ') as parts
WITH row,
    [p in parts where p contains 'address'] as address_part
WITH row,
    case when size(address_part) > 0 then substring(address_part[0], 10, size(address_part[0])-10) else
null end as address
WHERE address IS NOT NULL
```

```
MATCH (c:Client {client_id: toInteger(row.client_id)})
MATCH (e:Email {address: address})
MERGE (c)-[:USES]->(e)
RETURN count(*) as email_relationships;

// Client -> uses -> Device
:auto USING PERIODIC COMMIT 500
LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row
WITH row WHERE row.devices IS NOT NULL AND row.devices <> '[]' AND row.devices <> ''
WITH row, split(substring(row.devices, 1, size(row.devices)-2), '}, {') as device_strings
UNWIND device_strings as device_str
WITH row, device_str,
    case when device_str contains 'device_id' then true else false end as has_id
WHERE has_id
WITH row,
    replace(replace(replace(device_str, '{', ''), '}', ''), '"', '') as clean_str
WITH row, split(clean_str, ', ') as parts
WITH row,
    [p in parts where p contains 'device_id'] as id_part
WITH row,
    case when size(id_part) > 0 then substring(id_part[0], 11, size(id_part[0])-11) else null end as
device_id
WHERE device_id IS NOT NULL
MATCH (c:Client {client_id: toInteger(row.client_id)})
MATCH (d:Device {device_id: device_id})
MERGE (c)-[:USES]->(d)
RETURN count(*) as device_relationships;
```

9) Создание связей для GNN модели

```
// 6.1. Частота переводов между клиентами
MATCH (c1:Client)-[:OWNS]->(a1:Account)-[:SENT]->(t:Transaction)-[:RECEIVED]->(a2:Account)<-
[:OWNS]-(c2:Client)
WHERE c1 <> c2
```

```
WITH c1, c2,
     count(t) as transaction_count,
     sum(t.amount) as total_amount,
     avg(t.amount) as avg_amount,
     sum(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END) as fraud_count
MERGE (c1)-[r:TRANSFERRED_TO]->(c2)
SET
    r.transaction_count = transaction_count,
    r.total_amount = total_amount,
    r.avg_amount = avg_amount,
    r.fraud_count = fraud_count,
    r.fraud_ratio = toFloat(fraud_count) / transaction_count
RETURN count(r) as client_to_client_edges;

// 6.2. Подозрительное время транзакций
MATCH (c:Client)-[:OWNS]->(a:Account)-[:SENT]->(t:Transaction)
WITH c,
     count(t) as total_tx,
     sum(CASE WHEN t.tx_hour >= 0 AND t.tx_hour <= 5 THEN 1 ELSE 0 END) as night_tx,
     sum(CASE WHEN t.tx_hour >= 2 OR t.tx_hour <= 5 THEN 1 ELSE 0 END) as late_night_tx
SET
    c.night_transaction_ratio = toFloat(night_tx) / total_tx,
    c.late_night_transaction_ratio = toFloat(late_night_tx) / total_tx
RETURN count(c) as clients_updated;

// 6.3. Общие устройства/телефоны/email между клиентами
MATCH (c1:Client)-[:USES]->(x)<-[:USES]-(c2:Client)
WHERE c1 <> c2
WITH c1, c2,
     collect(DISTINCT labels(x)[0]) as shared_types,
     count(DISTINCT x) as shared_count
MERGE (c1)-[r:SHARES_CONTACT]->(c2)
SET
    r.shared_count = shared_count,
```

```
    r.shared_phones = size([t IN shared_types WHERE t = 'Phone']),
    r.shared_emails = size([t IN shared_types WHERE t = 'Email']),
    r.shared_devices = size([t IN shared_types WHERE t = 'Device'])
RETURN count(r) as shared_contact_edges;

// 6.4. Агрегированные метрики для счетов
MATCH (a:Account)
OPTIONAL MATCH (a)-[:SENT]->(t:Transaction)
WITH a,
     count(t) as outgoing_tx_count,
     sum(t.amount) as outgoing_total,
     avg(t.amount) as outgoing_avg,
     sum(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END) as outgoing_fraud
OPTIONAL MATCH (a)<-[:RECEIVED]-(t2:Transaction)
WITH a,
     outgoing_tx_count, outgoing_total, outgoing_avg, outgoing_fraud,
     count(t2) as incoming_tx_count,
     sum(t2.amount) as incoming_total,
     avg(t2.amount) as incoming_avg,
     sum(CASE WHEN t2.is_fraud = 1 THEN 1 ELSE 0 END) as incoming_fraud
SET
    a.outgoing_tx_count = outgoing_tx_count,
    a.outgoing_total = outgoing_total,
    a.outgoing_avg = outgoing_avg,
    a.outgoing_fraud = outgoing_fraud,
    a.incoming_tx_count = incoming_tx_count,
    a.incoming_total = incoming_total,
    a.incoming_avg = incoming_avg,
    a.incoming_fraud = incoming_fraud,
    a.total_tx_count = coalesce(outgoing_tx_count, 0) + coalesce(incoming_tx_count, 0),
    a.net_flow = coalesce(incoming_total, 0) - coalesce(outgoing_total, 0)
RETURN count(a) as accounts_updated;
```

10) Кол-во связей каждого типа

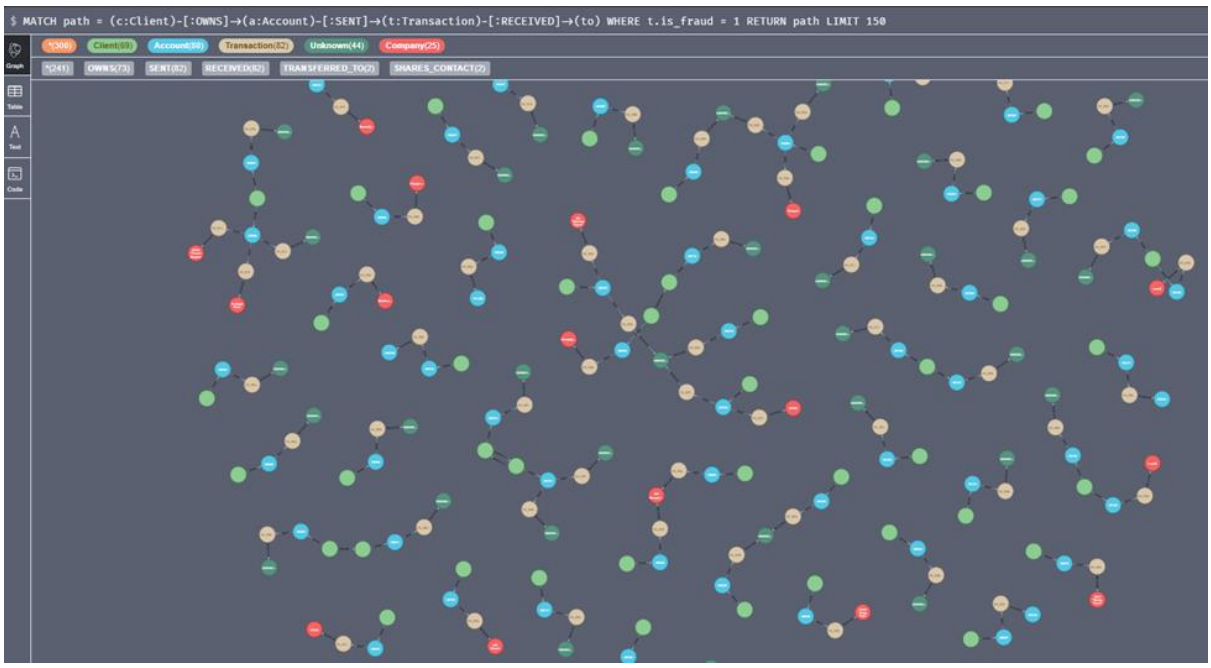
```
MATCH ()-[r:OWNS]->() RETURN 'OWNS' as RelationshipType, count(*) as Count

UNION ALL

MATCH ()-[r:ASSOCIATED_WITH]->() RETURN 'ASSOCIATED_WITH' as RelationshipType, count(*) as Count

UNION ALL

MATCH ()-[r:SENT]->() RETURN 'SENT' as RelationshipType, count(*) as Count

UNION ALL

MATCH ()-[r:RECEIVED]->() RETURN 'RECEIVED' as RelationshipType, count(*) as Count

UNION ALL

MATCH ()-[r:USES]->() RETURN 'USES' as RelationshipType, count(*) as Count

UNION ALL

MATCH ()-[r:TRANSFERRED_TO]->() RETURN 'TRANSFERRED_TO' as RelationshipType, count(*) as Count

UNION ALL

MATCH ()-[r:SHARES_CONTACT]->() RETURN 'SHARES_CONTACT' as RelationshipType, count(*) as Count

ORDER BY Count DESC
```

```
$ MATCH ()-[r:OWNS]→() RETURN 'OWNS' as RelationshipType, count(*) as Count UN…
```

Table | Text | Code

RelationshipType	Count
"OWNS"	102679
"ASSOCIATED_WITH"	2256
"SENT"	600000
"RECEIVED"	600000
"USES"	210083
"TRANSFERRED_TO"	389969
"SHARES_CONTACT"	99878

11) Визуализация цепочки мошеннических переводов



MATCH path = (c:Client)-[:OWNS]->(a:Account)-[:SENT]->(t:Transaction)-[:RECEIVED]->(to)

WHERE t.is_fraud = 1

RETURN path

LIMIT 150

Пример смурфинг – выгрузка как png – можно скачать сразу через Neo4j

MATCH (c:Client)-[:OWNS]->(a:Account)-[:SENT]->(t:Transaction)

WHERE t.amount < 50000

WITH c, count(t) as small_tx_count, sum(t.amount) as small_tx_total

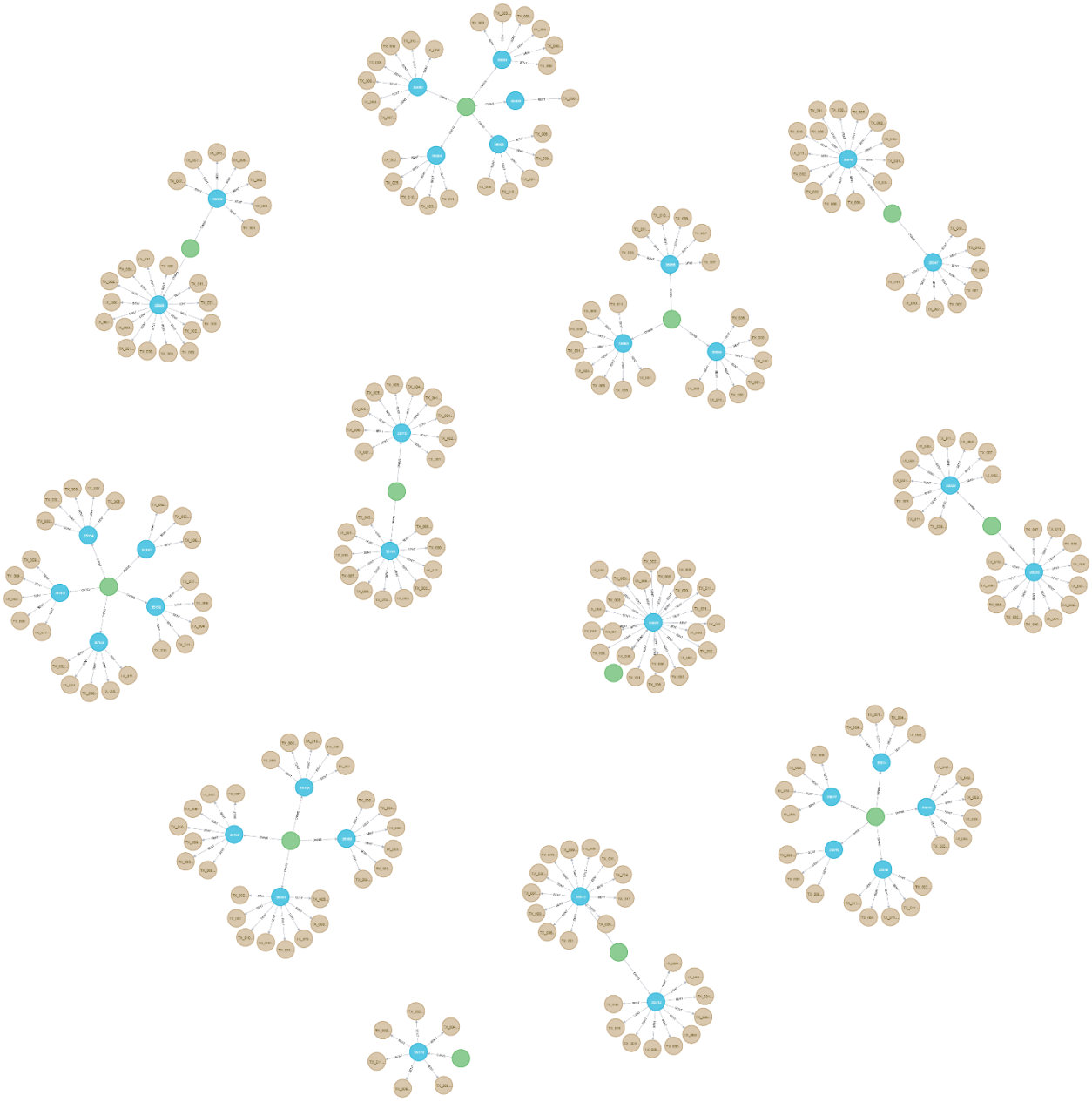
WHERE small_tx_count > 20

MATCH path = (c)-[:OWNS]->(a:Account)-[:SENT]->(t:Transaction)

WHERE t.amount < 50000

RETURN path

LIMIT 1000



12) Выгрузка nodes и edges

```
// Узлов + атрибуты (исключая null и пустые значения)
MATCH (n)
WHERE
    // последние 30 дней
    (labels(n)[0] <> 'Transaction' OR n.tx_datetime > datetime('2025-03-01T00:00:00'))
    // без Unknown
    AND labels(n)[0] <> 'Unknown'
RETURN
    id(n) as node_id,
    labels(n)[0] as node_type,
```

```
    CASE labels(n)[0]
        WHEN 'Client' THEN
            {client_id: n.client_id, fio: n.fio, risk_score: n.risk_score,
             client_tier: n.client_tier, registration_region: n.registration_region,
             night_transaction_ratio: n.night_transaction_ratio,
             late_night_transaction_ratio: n.late_night_transaction_ratio}
        WHEN 'Account' THEN
            {account_id: n.account_id, account_type: n.account_type,
             balance: n.balance, currency: n.currency, status: n.status,
             risk_score: n.risk_score, total_tx_count: n.total_tx_count,
             outgoing_total: n.outgoing_total, incoming_total: n.incoming_total}
        WHEN 'Company' THEN
            {company_id: n.company_id, name: n.name, activity_name: n.activity_name,
             inn: n.inn, risk_score: n.risk_score}
        WHEN 'Transaction' THEN
            {transaction_id: n.transaction_id, amount: n.amount,
             tx_datetime: n.tx_datetime, tx_hour: n.tx_hour,
             tx_day_of_week: n.tx_day_of_week, transaction_type: n.transaction_type,
             is_fraud: n.is_fraud, fraud_score: n.fraud_score, status: n.status,
             frequency_24h: n.frequency_24h}
        WHEN 'Phone' THEN
            {number: n.number, type: n.type, is_shared: n.is_shared}
        WHEN 'Email' THEN
            {address: n.address, domain: n.domain, is_shared: n.is_shared}
        WHEN 'Device' THEN
            {device_id: n.device_id, device_type: n.device_type,
             os: n.os, is_shared: n.is_shared}
        ELSE
            {id: n.id}
    END as properties
ORDER BY node_type, node_id

// Связи + атрибуты (исключая null и пустые значения)
MATCH (source)-[r]->(target)
```

```
WHERE
    // без Unknown связей
    labels(source)[0] <> 'Unknown' AND labels(target)[0] <> 'Unknown'
    // для транзакций фильтр по дате
    AND NOT (type(r) = 'SENT' AND source.tx_datetime < datetime('2025-03-01T00:00:00'))
    AND NOT (type(r) = 'RECEIVED' AND target.tx_datetime < datetime('2025-03-01T00:00:00'))
RETURN
    id(source) as source_id,
    labels(source)[0] as source_type,
    id(target) as target_id,
    labels(target)[0] as target_type,
    type(r) as relationship_type,
    CASE type(r)
        WHEN 'OWNS' THEN
            {ownership_type: 'direct'}
        WHEN 'ASSOCIATED_WITH' THEN
            {role: r.role}
        WHEN 'SENT' THEN
            {amount: source.amount, tx_datetime: source.tx_datetime}
        WHEN 'RECEIVED' THEN
            {amount: target.amount, tx_datetime: target.tx_datetime}
        WHEN 'USES' THEN
            {usage_type: 'contact'}
        WHEN 'TRANSFERRED_TO' THEN
            {transaction_count: r.transaction_count,
             total_amount: r.total_amount,
             avg_amount: r.avg_amount,
             fraud_count: r.fraud_count,
             fraud_ratio: r.fraud_ratio}
        WHEN 'SHARES_CONTACT' THEN
            {shared_count: r.shared_count,
             shared_phones: r.shared_phones,
             shared_emails: r.shared_emails,
             shared_devices: r.shared_devices}
```

```
        ELSE
            {}
    END as properties
ORDER BY source_type, source_id, relationship_type, target_type, target_id
```